

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandra sudevan

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Sequential File Organization vs Heap File Organization

Aspect	Sequential File Organization	Heap File Organization
Insertion	New records are usually inserted according to a specific order, so insertion can be slower.	Records can be inserted wherever free space is available, making insertion fast.
Deletion	Deleting a record may require maintaining the sequence of records.	Deletion is simple, although free spaces may be created.
Retrieval	Very efficient for sequential processing and ordered searches.	Efficient for scanning the complete file but slower for finding a particular record.
Searching	Can use the ordering to improve searching.	Usually requires a linear search if there is no index.
Best use	Suitable for reports and applications that process records in order.	Suitable for applications where records are frequently inserted.
Example	Student records arranged by Register Number.	Temporary transaction records stored without any particular order.

Analysis:

Sequential organization is better when records need to be accessed in a particular order. Heap organization is better when fast insertion is more important. Therefore, the choice depends on whether the application mainly performs **ordered retrieval or frequent insertion**.

Q2. Clustered Indexing vs Non-Clustered Indexing

Aspect	Clustered Indexing	Non-Clustered Indexing
Data arrangement	Determines the physical order of records.	Does not change the physical order of records.
Number	Usually only one clustered index per table.	Multiple non-clustered indexes can be created.
Range queries	Very efficient because related records are stored together.	Less efficient than clustered indexing for large range retrievals.
Storage	Index and data are closely related.	Requires additional index storage.
Example	Employee table clustered by Employee_ID.	Separate index on Employee_Name.

Main advantage	Fast retrieval of groups or ranges of records.	Provides fast searching using different columns.
-----------------------	--	--

Example:

Suppose an employee table is physically arranged according to Employee_ID. A clustered index on Employee_ID allows nearby employee records to be accessed efficiently. A non-clustered index on Employee_Name separately stores name entries and pointers to the actual records.

Analysis:

Clustered indexing is generally better for **range-based retrieval**, while non-clustered indexing provides flexibility when searching using different attributes.

Q3. Primary Indexing vs Secondary Indexing

Aspect	Primary Indexing	Secondary Indexing
Based on	Usually created using the primary key or ordering key.	Created on a non-ordering attribute.
Storage	Generally requires less index space, especially when sparse.	Often requires more storage because it may need more index entries.
Search performance	Very good for primary-key searches.	Good for searches on non-primary attributes.
Duplicates	Primary key values are unique.	Secondary index may contain duplicate values.
Example	Index on Student_ID.	Index on Department or Student_Name.
Use	Finding records using their unique identifier.	Finding records using other frequently searched attributes.

Analysis:

Primary indexing is efficient when the application frequently searches using a unique key. Secondary indexing improves flexibility but may require additional storage. For example, searching for a student using Student_ID is naturally suited to primary indexing, while searching students by Department can benefit from secondary indexing.

Q4. Static Hashing vs Dynamic Hashing

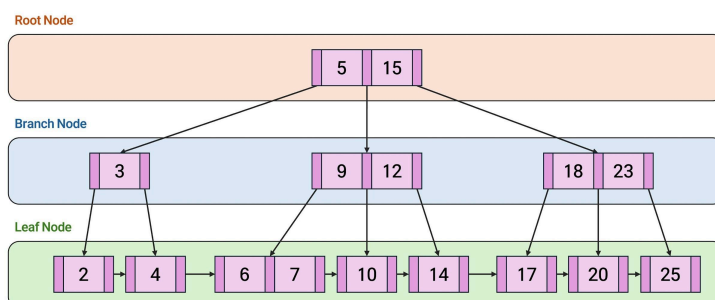
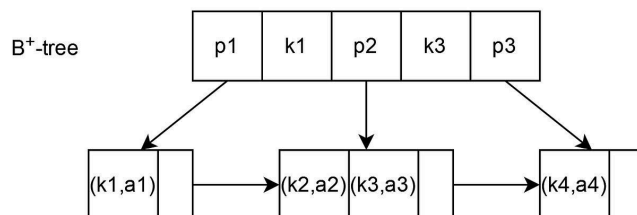
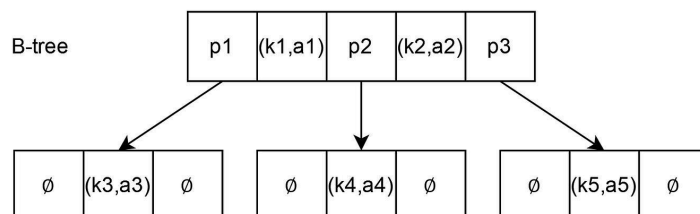
Aspect	Static Hashing	Dynamic Hashing
--------	----------------	-----------------

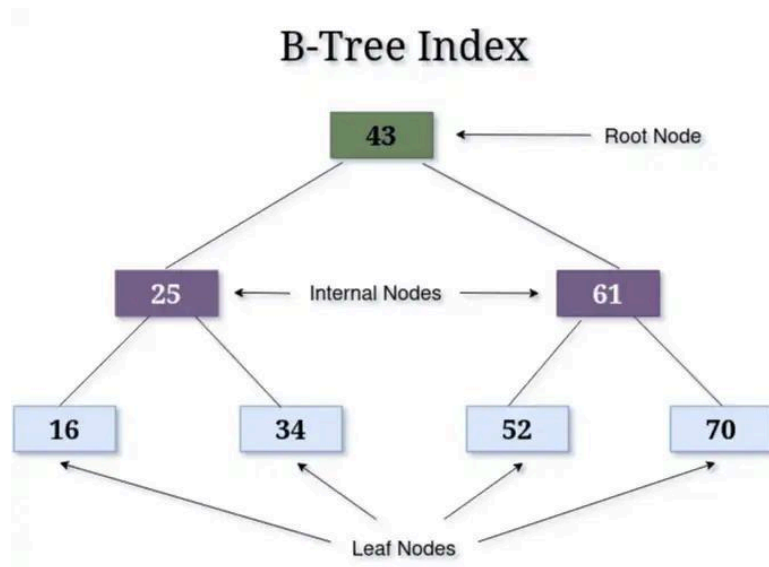
Number of buckets	Fixed when the file is created.	Can increase or decrease as data changes.
Scalability	Limited for growing databases.	Highly scalable.
Overflow	Overflow buckets may be required when buckets become full.	Handles growth by splitting buckets.
Performance	Can decrease when many overflow records occur.	Maintains better performance as the database grows.
Complexity	Simple to implement.	More complex to implement.
Example	Fixed hash table for a small student database.	Extendible hashing used for a growing customer database.

Analysis:

Static hashing works well when the number of records is relatively stable. However, when a database grows continuously, dynamic hashing is more suitable because it adjusts the bucket structure. Thus, **dynamic hashing provides better scalability**, while static hashing is simpler.

Q5. B-Tree Indexing vs B+ Tree Indexing





Aspect	B-Tree	B+ Tree
Data storage	Data pointers/records can exist in internal and leaf nodes	Actual record pointers are stored mainly at leaf nodes.
Leaf nodes	Usually not necessarily linked.	Leaf nodes are linked sequentially.
Range queries	Less efficient compared with B+ Tree.	Very efficient because leaf nodes are linked.
Search	Good search performance.	Excellent search and range-query performance.
Fan-out	Lower compared with B+ Tree in many implementations.	Higher because internal nodes store only keys and child pointers.
Database use	General-purpose indexing.	Widely preferred in database systems.

Analysis:

B+ Trees are generally preferred for large-scale databases because their linked leaf nodes make **range queries and sequential access faster**. Their higher fan-out can also reduce the height of the tree, resulting in fewer disk accesses.

Q6. Linear Hashing vs Extendible Hashing

Aspect	Linear Hashing	Extendible Hashing
--------	----------------	--------------------

Bucket growth	Buckets are split gradually in a linear order.	Buckets are split based on hash prefixes.
Directory	Does not require a directory.	Uses a directory containing bucket pointers.
Scalability	Good for gradually growing files.	Very good for dynamically growing databases.
Memory	Avoids directory overhead.	Requires memory for the directory.
Bucket management	Splitting occurs progressively.	Splitting is controlled by local and global depth.
Complexity	Relatively simple.	More complex due to directory management.

Analysis:

Linear hashing is useful when gradual expansion is preferred without maintaining a large directory. Extendible hashing can provide very fast access and flexible growth but requires additional directory management. Therefore, the better choice depends on the database's **growth pattern and memory requirements**.

Q7. Dense Indexing vs Sparse Indexing

Aspect	Dense Indexing	Sparse Indexing
Index entries	Contains an entry for every search-key value or record.	Contains entries for only some records.
Memory usage	Higher.	Lower.
Search speed	Faster because every record can have a direct index entry.	Slightly slower because additional searching may be required.
Maintenance	More entries must be updated when data changes.	Easier to maintain.
Storage requirement	Large.	Relatively small.
Use	Suitable when very fast direct searching is required.	Suitable for large, ordered files where memory is limited.

Analysis:

Dense indexing provides faster access but consumes more memory. Sparse indexing saves

memory but may require additional searching after reaching the nearest index entry. For very large databases, sparse indexing can be more storage-efficient.

Q8. Indexed Sequential File Organization vs Direct File Organization

Aspect	Indexed Sequential File	Direct File
Access method	Supports both sequential and indexed access.	Provides direct access using a calculated address.
Searching	Efficient using indexes.	Very fast for exact-match searches.
Range queries	Good for range queries.	Usually less suitable for range queries.
Storage	Requires additional index storage.	Does not require a large index structure.
Complexity	More complex because indexes must be maintained.	Simpler access mechanism.
Best use	Applications requiring both sequential and random access.	Applications requiring fast exact record retrieval.

Analysis:

Indexed sequential organization provides greater flexibility because it supports both sequential and indexed access. Direct organization is usually faster for exact-match searches but is less suitable for ordered or range-based retrieval. Therefore, indexed sequential organization is useful for **mixed workloads**, while direct organization suits **exact lookups**.

Q9. Single-Level Indexing vs Multi-Level Indexing

Aspect	Single-Level Indexing	Multi-Level Indexing
Index structure	Contains one index level.	Contains multiple levels of indexes.
Search	Search is performed within one index.	Higher levels help locate the required lower-level index quickly.
Large databases	May become inefficient when the index is very large.	More suitable for very large databases.
Memory	Requires less structural overhead.	Requires additional index levels.

Access speed	Good for small or moderate indexes.	Efficient for large indexes.
Example	Simple index on a small student table.	B-Tree/B+ Tree style hierarchical indexing.

Analysis:

Single-level indexing is simple and works well for smaller databases. As the index becomes larger, searching the entire index becomes inefficient. Multi-level indexing reduces the search space at each level and therefore provides better performance for **large-scale database retrieval**.

Q10. Hashing Techniques vs Indexing Techniques

Aspect	Hashing	Indexing
Exact-match queries	Excellent performance for exact matches.	Very good performance.
Range queries	Generally poor because records are distributed across buckets.	Excellent, especially with B+ Trees.
Data organization	Uses a hash function and buckets.	Uses structures such as B-Trees/B+ Trees.
Scalability	Dynamic hashing handles database growth well.	Multi-level indexes can handle large databases effectively.
Memory	Requires bucket and overflow management.	Requires storage for index structures.
Example query	WHERE Student_ID = 105	WHERE Student_ID BETWEEN 100 AND 200

Performance Evaluation

For an **exact-match query**, hashing is usually the better choice because the hash function can directly identify the bucket containing the required record.

For example:

```
SELECT * FROM Student
WHERE Student_ID = 105;
```

A hash-based structure can quickly locate the bucket for Student_ID = 105.

For a **range query**, indexing—especially B+ Tree indexing—is usually better:

```
SELECT * FROM Student  
WHERE Student_ID BETWEEN 100 AND 200;
```

A B+ Tree can locate the starting value and then traverse the linked leaf nodes to retrieve the remaining records efficiently.

Final Comparison

Hashing → Best for exact-match queries

B+ Tree Indexing → Best for range and ordered queries

Therefore, there is no single technique that is best for every situation. The database designer should select the technique based on the type of queries, data growth, storage limitations, and required retrieval speed.

ention, while sequential files are better for ordered processing. Clustered and B+ Tree indexes.